



UNIVERSITY OF BUEA

Faculty Of Engineering and Technology

CEF440

Internet Programming and Mobile Programming

Group 16

TASK 5: UI DESIGN AND IMPLEMENTATION

Submitted to:

Dr. Nkemeni Valery

2025/2026

GROUP MEMBERS

NKENG SAMA MOKOM	FE23A118
ENOW JOHN EYONG	FE23A046
CHIANGOIESEH CHRIS RAWLINGS	FE23A029
KEUKANG CARMEN ALICIA	FE23A074
NKENGAFAC WILLYHERMAN FONKEM	FE23A119

Table of Contents

1. INTRODUCTION.....	4
2. APP IDENTITY	4
2.1. Name and Concept.....	4
2.2. Mission Statement.....	5
2.3. Iconography and Logo Design.....	5
2.4. Brand Colour Palette	5
2.5 Typography	6
3. VISUAL DESIGN.....	6
3.1. Design Methodology.....	6
3.2. UI Components and Screen Flow	7
3.3 Screen Designs.....	8
3.3.1 Splash Screen and Onboarding	8
3.3.2 Dashboard Screen	9
3.3.3 Feedback Prompt (Modal).....	9
3.3.4 History Screen.....	10
3.3.5 Map Screen.....	10
3.3.6 Settings Screen.....	10
4. FRONTEND IMPLEMENTATION	11
4.1 Tool Used: React Native	11
4.2 Development Approach	11
4.3 Key Implemented Features and Screens	13
4.4 UI/UX Implementation Considerations	14
5. TOOLS SUMMARY	14
6. CONCLUSION	15

1. INTRODUCTION

This phase presents the UI design and implementation of our mobile application developed for the real-time collection of Quality of Experience (QoE) data from mobile network subscribers. The application is designed to run continuously in the background, periodically prompting users for subjective feedback while passively capturing objective network and device performance metrics.

The user interface is a critical layer of the system, as it mediates all direct interactions between the subscriber and the data collection engine. A well-designed UI ensures that users are willing to engage with feedback prompts, understand the purpose of the application, and continue using it over time without frustration. Poor interface design, conversely, leads to abandonment, low data quality, and unreliable longitudinal datasets.

This part documents the full UI design lifecycle: from the establishment of brand identity and visual language, through to the implementation of production-ready frontend components. It covers the design system, screen architecture, accessibility considerations, and the technical frameworks used to bring the designs to life on Android devices targeting subscribers in Cameroon and similar developing-country contexts.

2. APP IDENTITY

App Identity defines the core personality, visual signature, and user perception of the application. For **NetInsight**, the identity is structured around transparency, reliability, and effortless utility. The application functions as a passive network probe and a subjective survey medium, meaning its identity must feel lightweight, professional, and trustworthy to justify background resource access.

2.1. Name and Concept

The application is named **NetInsight**, a compound of 'network' and 'insight', it signifies the convergence of objective network metrics (telemetry) with subjective user experience (Quality of Experience - QoE). By emphasizing "Experience," the brand shifts focus from raw machine performance data to the human-centric reality of digital connectivity within developing regions like Cameroon.

2.2. Mission Statement

"Empowering mobile subscribers to share their real network experience automatically, honestly, and effortlessly."

2.3. Iconography and Logo Design

The central icon represents systemic monitoring combined with telecommunication insights.

- **The Outer Eye Ring:** Represents visibility, real-time observation, and the continuous oversight of network parameters in the background.
- **The Data/Signal Waves:** Built into the core of the icon, these emphasize mobile data packets, signal strength telemetry, and cellular performance logs.
- **The Teal Accent Color (\$#00D2C4\$):** Chosen specifically to give a modern, clean tech vibe that balances out the deep administrative blue, signaling real-time activity and high performance.



2.4.Brand Colour Palette

The color scheme is designed to maximize visual comfort, distinguish application zones, and match a clean engineering aesthetic.

Color Token	Natural Colors	Hex Code	Applied System Context
Primary Background	Dark Sapphire Blue	#173F7A	Applied globally to onboarding, language selection, and core workspace backgrounds. Provides high contrast for white text elements.
Card / Widget Background	Cobalt Blue	#1B4A8F	Applied to dashboard analytics modules, log containers, and legend modules to create visual hierarchy.

Teal Branding Accent	Robin's Egg Blue	#00D2C4	Used exclusively for primary logo elements, active language button borders, and focal points.
Muted Blue Typography	Ice Blue	#A0B2CE	Applied to secondary data labels, subtitles, and inactive tab properties to minimize cognitive load.
Static Light Gray	Alice Blue	#E0E6ED	Used for early-stage selection layouts (e.g., non-active language buttons) to provide a neutral base.
Coverage Indicators	Emerald Green/ Light Red/ Sunflower Yellow	#2ECC71 / #F1C40F / #E74C3C	Standardized Green, Yellow, and Red nodes indicating Excellent, Fair, and Bad network coverage respectively.

2.5 Typography

- **Scale Strategy:** Bold, large layout headers (22px - 24px) are anchored uniformly across both modal and static screens. Subheadings and metric card parameters are held at a readable 12px - 14px scale.
- **Layout Geometry:** The application strictly follows a card-based architecture with border-radii ranging between 10px and 14px. This ensures the UI elements feel modern and prevents visual fatigue across crowded analytical dashboards.

3. VISUAL DESIGN

The visual design system ensures high legibility, accessible interface patterns, and structural consistency across the onboarding flow and the main application workspace.

We used Figma for our Visual Design.

3.1. Design Methodology

The visual design was developed using a component-driven, mobile-first methodology aligned

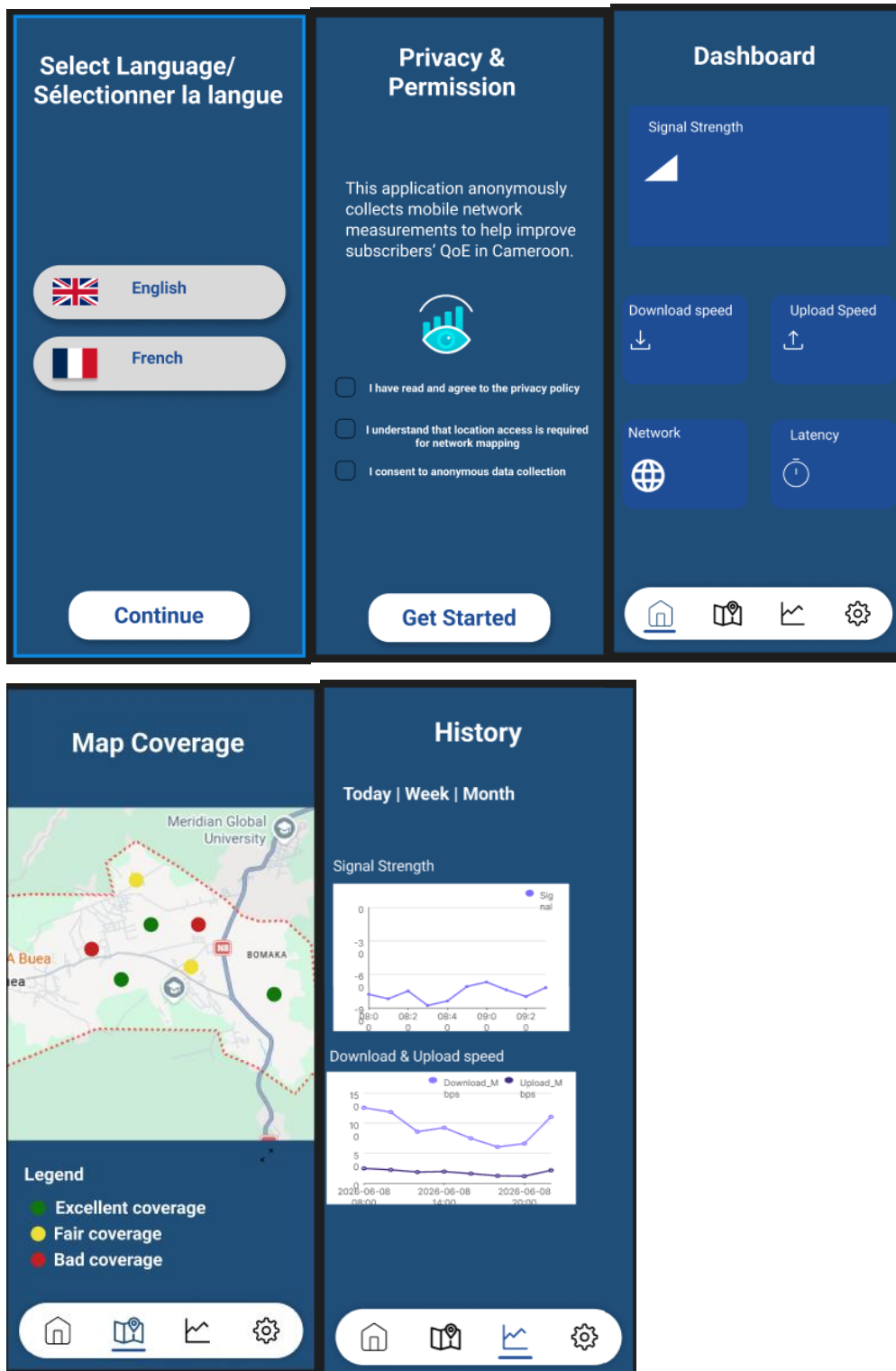
with the Material Design 3 (M3) design system. Figma was used as the primary design tool, enabling the creation of a shared component library, auto-layout frames, and interactive prototype flows for user testing prior to implementation.

The design process followed an iterative cycle: requirements analysis, information architecture, lo-fi wireframing, hi-fi mockup, prototype, review, and refinement. Each iteration was evaluated against usability heuristics and the specific needs of mobile network subscribers who may have varying levels of digital literacy.

3.2. UI Components and Screen Flow

Based on the design layouts, the user journey is divided into a 7-screen interactive flow optimized to secure explicit user permissions before data logging begins:

1. **Language Selection Screen:** A clean bilingual gateway containing stacked target buttons for English and French selection, utilizing the accent teal border to mark user preference.
2. **Privacy & Permission Screen:** A detailed legal and technical disclosure screen. It requires the explicit activation of three distinct checkbox nodes covering Privacy Terms, Background Location mapping, and Anonymous Telemetry tracking before enabling the entry path.
3. **Dashboard Screen:** The core user interface, displaying an executive visualization summary area followed by a 2 x 2 matrix grid reporting real-time objective parameters: Download Speed, Upload Speed, Network Type, and Latency.
4. **Map Coverage Screen:** A spatial visualization layer that simulates geographical tracking over an area (e.g., *Meridian Global University*), plotting live color-coded cellular coverage nodes against a structured visual map legend.
5. **Network History Screen:** A timeline interface with time-range filtering controls ("Today", "Week", "Month") that lets subscribers track background telemetry logs and operator performance drops (e.g., *MTN Cameroon, Orange Cameroun*).
6. **Settings Screen:** A system management terminal displaying background passive runtime flags and telemetry prompt window intervals (e.g., *Every 4 Hours*).



3.3 Screen Designs

3.3.1 Splash Screen and Onboarding

The splash screen displays the NetInsight full lockup logo centred on the brand-primary

background with a fade-in animation. On first launch, users are directed to a three-screen onboarding flow that explains the app's purpose, the types of data collected, and the notification permissions required.

The onboarding cards use illustrative icons with brief explanatory copy (maximum 25 words per screen). A prominent 'Allow Notifications' CTA on screen 3 is pre-requisite to enabling the feedback prompt system. A 'Skip' option is present but subtly styled to discourage premature skipping without being obstructive.

3.3.2 Dashboard Screen

The dashboard is the primary screen encountered after onboarding. It is designed to provide at-a-glance understanding of current network conditions without requiring user interaction. Key design elements include:

- Network Status Card: displays current network type (5G/4G/3G/2G), signal bars, operator name, and a colour-coded status badge (Excellent / Good / Fair / Poor)
- Metric Tile Grid: a 2x3 grid of compact tiles showing Signal Strength (dBm), Latency (ms), Jitter (ms), Packet Loss (%), Download Speed (Mbps), and Battery/CPU usage
- Last QoE Score Panel: shows the most recent user-submitted satisfaction score with timestamp and a sparkline of the last 7 sessions
- Background Service Indicator: a subtle pulsing dot in the top-right of the screen header confirming that monitoring is active

3.3.3 Feedback Prompt

The feedback prompt is the most interaction-critical screen in the application. It is triggered by a background service notification and presented as a bottom sheet modal to minimise context disruption. Design decisions include:

- Maximum 4 interaction steps per session to limit fatigue
- Step 1 — Overall Satisfaction: a 5-point emoji-anchored slider (from 'Very Dissatisfied' to 'Very Satisfied')
- Step 2 — Dimension Ratings: three compact horizontal star-rating rows for Call Quality, Data Speed, and App Responsiveness
- Step 3 — Optional Comment: a single-line open text field with placeholder 'Describe your experience (optional)'

- Step 4 — Confirmation: an animated checkmark with 'Thank you, your feedback was recorded' and session stats

Progress is indicated by a 4-dot stepper at the top of the sheet. 'Not now' is available at all steps and dismisses the modal without penalty, preserving user autonomy.

3.3.4 History Screen

The history screen presents a chronological list of data collection sessions. Each list item is a card showing: date/time, network type, overall QoE score (colour-coded), and a compact metric summary bar. Tapping any item opens a detail view with the full session data, including all metric readings and any submitted comment.

A filter chip row at the top allows filtering by date range, network type, and QoE score range. An export button (CSV) is present in the action bar for power users.

3.3.5 Map Screen

The map screen uses Google Maps SDK with a custom heatmap overlay layer powered by collected QoE and signal data. Colour gradients range from red (poor signal/QoE) through amber to green (strong signal/QoE). Cluster markers show aggregated scores in areas with multiple readings. Users can toggle between Signal Strength mode and QoE Score mode.

3.3.6 Settings Screen

The settings screen is structured into logical groups using M3 list item patterns with section headers:

- Notifications: toggle prompts on/off, set frequency (daily/twice daily/weekly), set quiet hours
- Privacy & Data: view data sharing consent status, link to privacy policy, option to delete all local data
- Account: subscriber ID display, opt-out option, app version
- Accessibility: font size override, high contrast mode, reduce motion toggle

4. FRONTEND IMPLEMENTATION

The frontend of the mobile application was implemented using React Native, a popular framework for building cross-platform mobile applications.

4.1 Tool Used: React Native

Reasons for Selecting React Native

- Cross-platform development: Enables a single codebase for both Android and iOS
- Performance efficiency: Provides near-native performance using native components
- Component-based architecture: Encourages modular and reusable UI components
- Large ecosystem: Strong community support and availability of libraries
- Hot reload feature: Speeds up development by instantly reflecting code changes

4.2 Development Approach

The implementation followed a structured component-based architecture:

- Creating the React Project: We created the react project from our code editor(VS Code) by accessing the terminal, navigating to the directory where we want to create the project then running the command “npx create-expo-app@latest” then we were prompted to enter our App name which we entered NetInsight the we went ahead to choose our SDK version where we selected SDK 54 cause in works in reference with Expo Go on Ios and Android.
- We then went ahead to create the different modules of our app in the “src/screens” folder in our directory and created other files for the program to work effectively.
- We then went ahead to install all the necessary node.js packages for our project using “npm install” where it installed successfully.
- We moved on to install Expo packages too in our project by using npm install-expo” command.
- After successfully installing all the packages, we went on to start our React native project by using the command ‘npx expo start’ which our project started and we were able to view it by scanning the QR code displayed to open on React App on Expo Go on Android and iOS devices to test the functionality our our app which worked perfectly.
- UI Components: Reusable elements such as buttons, cards, and input fields
- Screens: Each screen implemented as a separate React component

- Navigation: Managed using React Navigation for smooth screen transitions
- State Management: Local and global state handling for user inputs and session data
- API Integration: Prepared for data submission

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\HP\Documents\L400_2nd_Semester\CEF440> cd C:\Users\HP\Documents\L400_2nd_Semester
PS C:\Users\HP\Documents\L400_2nd_Semester> npx create-expo-app@latest
✓ What is your app named? ... NetInsight
✓ Select an Expo SDK version: » For learning with Expo Go (SDK 54)
Creating NetInsight using the default template.

Tip:
  * npx create-expo-app --template to pick from other templates
  * npx create-expo-app --example to explore https://github.com/expo/examples

✓ Downloaded and extracted project files.
> npm install
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce
e async requests by a key value, which is much more comprehensive and powerful.
Explore and understand your code
  
```

```

PS C:\Users\HP\Documents\L400_2nd_Semester\NetInsight> npx expo start
Starting project at C:\Users\HP\Documents\L400_2nd_Semester\NetInsight
React Compiler enabled
Starting Metro Bundler

```



Ln 10, Col 2 Spaces: 2 UTF-8 LF { } JavaScript Go Live Prettier

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\HP\Documents\L400_2nd_Semester\NetInsight> npx expo start

> Metro waiting on exp://172.16.210.253:8081
> Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

> Web is waiting on http://localhost:8081

> Using Expo Go
> Press s | switch to development build

> Press a | open Android
> Press w | open web

> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
  
```

Ln 10, Col 2 Spaces: 2 UTF-8 LF { } JavaScript Go Live Prettier

```

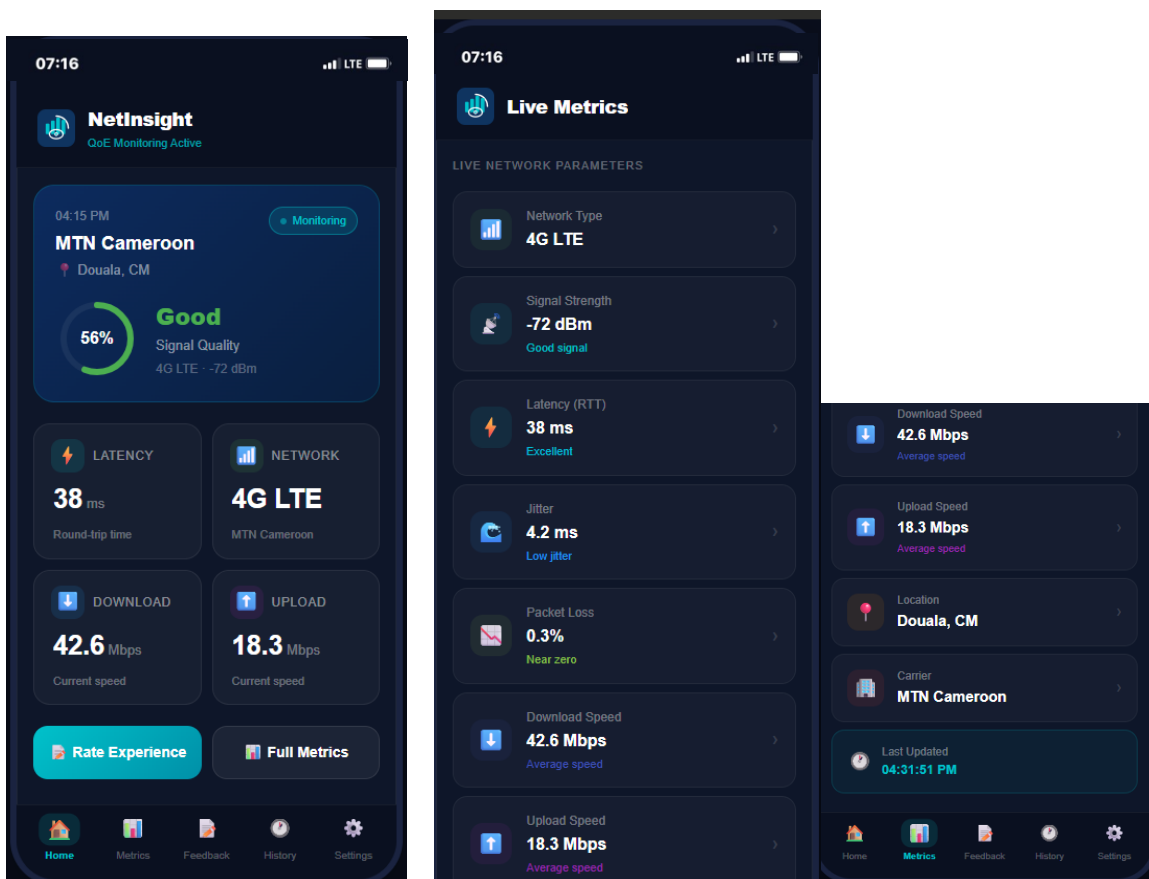
> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor

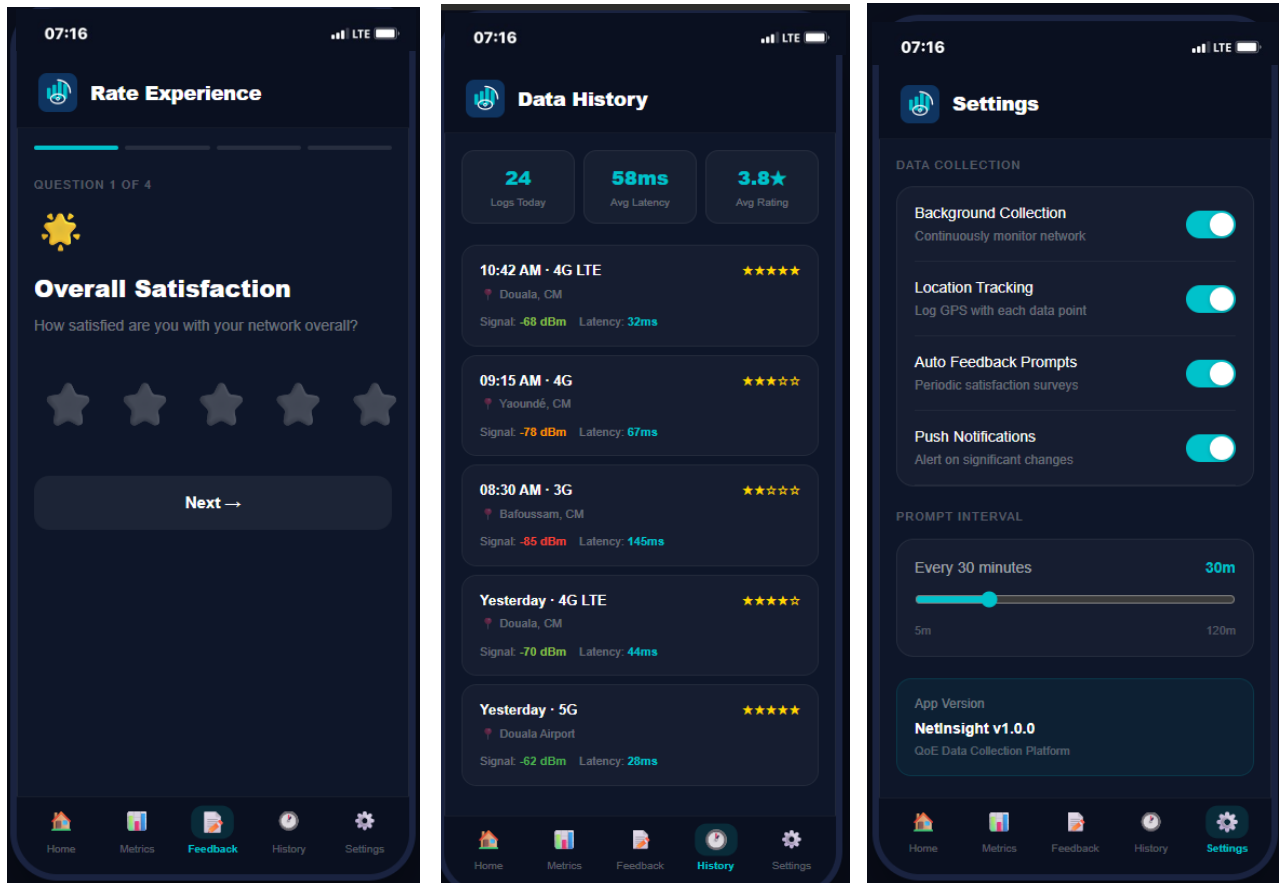
> Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.
  
```

4.3 Key Implemented Features and Screens

- User onboarding.
- Feedback input forms for user experience reporting
- Network quality rating system (e.g., signal strength, data speed perception)
- Navigation between multiple screens
- Basic form validation for user input accuracy





4.4 UI/UX Implementation Considerations

- Ensured consistency with Figma design specifications
- Maintained responsive layouts across different screen sizes
- Optimized touch targets for mobile usability
- Used reusable components to reduce redundancy and improve maintainability

5. TOOLS SUMMARY

Tool	Purpose	Reason for Use
Figma	UI/UX Visual design and prototyping	Collaboration, prototyping, design consistency, industry standard
React Native	Frontend development	Cross-platform support, performance, reusable components

6. CONCLUSION

The UI design and implementation phase successfully translated conceptual designs into a functional mobile application interface. Figma enabled the creation of a structured and user-centered visual design, while React Native provided an efficient framework for implementing a responsive and scalable frontend. The combination of these tools ensured consistency between design and implementation, resulting in a usable and maintainable mobile application interface for collecting user experience data.